CHANDIGARH UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MINI PROJECT REPORT

SPACE MISSION\nSIMULATOR

An Object-Oriented Programming Project in C++

Course: Object-Oriented Programming Using C++ | Course Code: 25CAH-201

SUBMITTED BY	SUBMITTED TO
Name: Dhiraj Ahlawat UID: 25bcd10162 Section: 25bcd-2(b) Programme: BCA (Data Science)	Faculty Name: Mr. Jitender Department: BCA Chandigarh University

ACADEMIC SESSION 2026–2027

Project Domain: Aerospace Mission Simulation & Resource Telemetry

1. INTRODUCTION & PROBLEM STATEMENT

1.1 Introduction

Space Mission Simulator is an interactive, text-based C++ application that models selected operations of a manned space voyage. A mission is represented through a spacecraft, a celestial destination, a crew of specialized astronauts and a sequence of travel stages. The simulator tracks important resources such as fuel, oxygen and hull integrity while responding to space events.

The project is developed using modern C++17 and demonstrates how Object-Oriented Programming can represent a complex real-world domain through modular classes, encapsulated data and specialized behavior. Destinations include the Moon, Mars, the International Space Station (ISS) and Europa.

1.2 Problem Statement

A space mission involves several interacting entities and operational constraints. A procedural design can become difficult to maintain when resource management, crew responsibilities, mission stages and emergency handling are mixed together. The project addresses this design challenge by building a modular simulation in which each major entity has a clear responsibility and interactions are coordinated by a Mission object.

1.3 Motivation

- Apply classroom OOPS concepts to a practical C++ application.
- Represent spacecraft resources and mission state with controlled access.
- Model different astronaut roles and hazards through polymorphism.
- Demonstrate safe memory ownership and persistent mission data.

1.4 Simulation Overview

A voyage progresses through four stages: Earth Orbit Departure, Deep Space Insertion, Mid-Course Cruise, and Destination Approach and Orbital Capture. Resource consumption and random events affect mission progress. The mission can complete successfully or fail if a critical resource is depleted.

Input / Entity	Role in simulation
Spacecraft	Maintains fuel, oxygen, hull integrity and subsystem condition.
Destination	Provides distance, required fuel and difficulty parameters.
Astronaut crew	Contributes role-specific navigation, repair or research skills.
Space events	Introduce hazards that can damage the spacecraft or consume resources.

2. OBJECTIVES, SCOPE & FEATURES

2.1 Project Objectives

- Build a modular C++17 application with separate interface, domain, subsystem and persistence responsibilities.
- Encapsulate spacecraft telemetry and enforce resource boundaries through member functions.
- Use abstract base classes and runtime polymorphism for astronauts and space events.
- Validate mission readiness before launch and evaluate failure conditions during travel.
- Save and restore mission data using file handling and move semantics.
- Generate mission summaries through a dedicated reporting module.

2.2 Scope

The current application is a terminal-based academic simulator. It focuses on discrete mission stages, resource management, crew composition, emergency events and save/load operations. It is intended for learning, demonstration and viva examination; it is not a physical flight dynamics or operational aerospace control system.

2.3 Implemented Features

Feature	Description
Mission setup	Create a mission and select a destination and spacecraft.
Crew management	Add up to four crew members with specialized roles.
Pre-launch checks	Check fuel, oxygen, hull integrity and healthy crew readiness.
Stage-based travel	Advance through mission stages while consuming resources.
Random events	Generate asteroid, oxygen leak, fuel leak and solar storm events.
Crew countermeasures	Relevant crew skills can reduce damage or restore resources.
Persistence	Save and load mission state through text-file serialization.
Mission analytics	Generate standard or detailed textual reports.

2.4 Future Enhancements

Potential extensions include a graphical user interface, richer destination models, more detailed physics, expanded event scenarios and automated visualization of telemetry. These are proposed improvements and are not represented as current implemented features.

3. TECHNOLOGIES, REQUIREMENTS & ARCHITECTURE

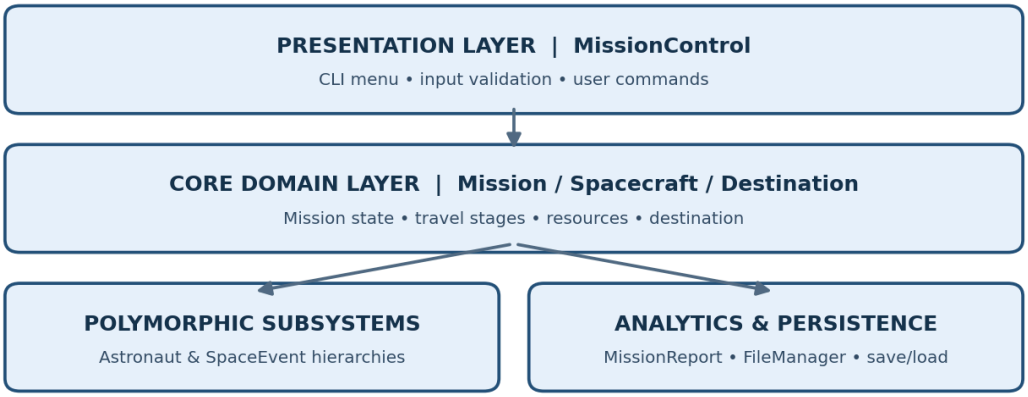
3.1 Technology Stack

Component	Technology / Specification
Programming language	C++17
Compiler	GCC 9.0+ or Apple Clang 15.0+
Build system	GNU Make
Build flags	-std=c++17 -Wall -Wextra -I./include
Supported environments	macOS, Linux, or Windows through WSL2
Persistence	C++ stream-based flat-file I/O

3.2 Hardware Requirements

The source report specifies a minimum of 512 MB RAM and approximately 50 MB storage. The application is designed to run efficiently on modern multi-core workstations.

3.3 Layered Architecture



Modular separation keeps interface, simulation rules, specialized behavior and file operations distinct.

Figure 1. Four-tier logical architecture of the Space Mission Simulator.

3.4 Project Directory Structure

```
SpaceMissionSimulator/\n├── main.cpp\n├── Makefile\n├── include/   # class and module declarations\n├── src/       # C++
method implementations\n└── data/      # mission save and verification files
```

The entry point creates MissionControl and calls run(). Header files declare the classes, while source files contain their implementations. The Makefile compiles the source files using the C++17 toolchain.

4. SYSTEM DESIGN & WORKING

4.1 Main Modules

Module	Responsibility
MissionControl	Displays menus, accepts validated input and routes user commands.
Mission	Coordinates craft, destination, crew, events, stages and mission state.
Spacecraft	Controls fuel, oxygen, hull integrity and resource operations.
Destination	Stores destination name, distance, fuel requirement and difficulty.
Astronaut hierarchy	Defines the abstract crew contract and specialized crew roles.
SpaceEvent hierarchy	Defines common hazard behavior and concrete event types.
FileManager	Serializes and reconstructs mission state from files.
MissionReport	Produces mission summary and detailed analytics.

4.2 Execution Flow

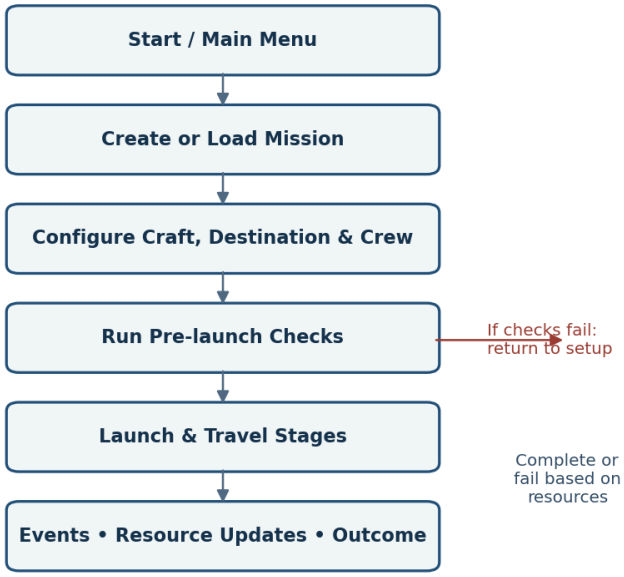


Figure 2. Simplified program execution and mission lifecycle.

4.3 Pre-launch Validation

Before launch, the system verifies that the spacecraft has sufficient fuel for the selected destination, oxygen is available, hull integrity is above zero, and at least one healthy crew member is present. The mission enters READY state only when all checks pass. Crew configuration also requires essential Commander, Pilot and Engineer roles.

4.4 Travel and Mission Outcome

During travel, the stage counter advances, fuel and oxygen are consumed, and event handling may affect resources. Failure conditions are checked after updates. Reaching the final stage with critical resources remaining results in mission completion; depletion of a critical resource can cause mission failure.

5. OBJECT-ORIENTED PROGRAMMING CONCEPTS

5.1 Classes and Objects

A class combines data and functions into a user-defined type. Spacecraft, Destination and Mission are examples of classes. A particular spacecraft or mission created during execution is an object.

5.2 Encapsulation

Spacecraft stores important telemetry such as fuel, oxygen and hull integrity. The class exposes controlled operations such as consumeFuel(), consumeOxygen(), takeDamage() and repairHull(), rather than requiring other modules to modify internal state directly.

5.3 Abstraction

Astronaut and SpaceEvent define common interfaces for families of related objects. The rest of the application can work with the abstract interface without depending on every concrete implementation detail.

5.4 Inheritance

Commander, Pilot, Scientist and Engineer specialize the Astronaut base class. AsteroidEvent, OxygenLeakEvent, FuelLeakEvent and SolarStormEvent specialize the SpaceEvent base class.

5.5 Polymorphism

The abstract classes declare virtual behavior. Calls made through base-class pointers or references are dispatched to the concrete derived implementation at runtime. This allows the mission to manage different crew roles and event types through common interfaces.

5.6 Additional C++ Features

Concept	Project use
Pure virtual functions	Astronaut duty and SpaceEvent behavior contracts.
Function overloading	MissionReport supports standard and detailed report generation.
Operator overloading	Spacecraft defines operator> for condition comparison.
Smart pointers	std::unique_ptr manages dynamically allocated crew and mission ownership.
Move semantics	Mission state can be transferred during file-load reconstruction.
Exception handling	Invalid input and malformed save data are handled safely.
File handling	Stream operations save and restore mission telemetry.

These concepts are used to improve modularity, maintainability, resource safety and extensibility.

6. CLASS DESIGN & IMPORTANT SOURCE CODE

6.1 Key Class Relationships

Class / hierarchy	Important members or behavior
Spacecraft	consumeFuel(), consumeOxygen(), takeDamage(), repairHull(), operator>()
Destination	getName(), getDistance(), getRequiredFuel(), getDifficulty()
Astronaut (abstract)	performDuty() and role/skill accessors
Commander / Pilot / Scientist / Engineer	Specialized astronaut roles and skills
SpaceEvent (abstract)	trigger(Spacecraft&) and displayDescription()
Concrete event classes	AsteroidEvent, OxygenLeakEvent, FuelLeakEvent, SolarStormEvent
Mission	performPreLaunchChecks(), launch(), advanceTravel(), checkFailureConditions()
FileManager	saveMission(), loadMission(), parsing and validation
MissionReport	generate() and generate(bool detailed)

6.2 Program Entry Point

```
#include "include/MissionControl.h"\n\nint main() {\n    MissionControl app;\n    app.run();\n    return 0;\n}
```

6.3 Pre-launch Check Logic (Excerpt)

```
bool Mission::performPreLaunchChecks() {\n    bool passed = true;\n    if (spacecraft.getFuel() >=
destination.getRequiredFuel())\n        std::cout << "Fuel: OK\\n";\n    else { passed = false; }\n\n    if
(spacecraft.getOxygen() <= 0 ||\n        spacecraft.getHullIntegrity() <= 0) passed = false;\n\n    bool hasHealthyCrew =
false;\n    for (const auto& mem : crew)\n        if (mem->getHealth() > 0) hasHealthyCrew = true;\n\n    if (passed &&
hasHealthyCrew) {\n        state = MissionState::READY;\n        return true;\n    }\n    return false;\n}
```

Note: The excerpts are shortened for report readability. The complete implementation remains in the project source files.

7. EVENT HANDLING, OUTPUT & PERSISTENCE

7.1 Space Event Handling

Event	Effect / crew response
Asteroid field	Consumes evasive fuel; collision damage is reduced when evasion succeeds. Pilot skill supports navigation.
Oxygen leak	Reduces oxygen; an Engineer can contain the leak and recover part of the oxygen.
Fuel leak	Reduces fuel; an Engineer can patch the line and recover part of the fuel.
Solar storm	Damages hull integrity; a Scientist can analyze telemetry and restore part of the hull.

7.2 Representative Terminal Interaction

```
SPACE MISSION SIMULATOR\n1. Create New Mission\n2. Manage Crew\n3. Launch / Continue Mission\n4. View Mission Report\n5. Save Mission\n6. Load Mission\n\nPRE-LAUNCH CHECKS\nFuel: OK\nOxygen: OK\nHull: OK\nCrew: OK\nPRE-LAUNCH: PASSED
```

The terminal block above is an illustrative layout based on documented application behavior, not a captured runtime screenshot. Actual wording and menu numbering may vary with the current source build.

7.3 Save and Load

FileManager writes mission information to a text file using key-value records. The documented fields include mission name, destination, state, stage, fuel, oxygen and hull values. Crew records store role, name and health. During loading, values are parsed and validated before the reconstructed mission is transferred into place.

```
file << "MISSION_NAME=" << mission.getMissionName() << "\\n"\n     << "DESTINATION=" << mission.getDestination().getName() << "\\n"\n     << "FUEL=" << mission.getSpacecraft().getFuel() << "\\n"\n     << "OXYGEN=" << mission.getSpacecraft().getOxygen() << "\\n";
```

7.4 Error Handling

The project checks input validity and rejects malformed or out-of-range save data. The source report documents a corrupt-save test in which invalid spacecraft data is detected and loading is aborted cleanly.

8. TESTING & EXPERIMENTAL RESULTS

The technical project report documents a test set covering compilation, mission setup, resource boundaries, crew validation, travel, events, mission outcomes and persistence. The table below summarizes representative test cases from that report.

Test	Operation	Documented result
TC-01	Clean compilation using make	PASS — binary generated with no reported warnings.
TC-03	Out-of-range craft values	PASS — hull capped at 100 and negative fuel clamped.
TC-04	Add a fifth crew member	PASS — maximum crew size enforced.
TC-05	Launch without essential roles	PASS — invalid crew configuration rejected.
TC-06 / 07	Pre-launch success and insufficient fuel	PASS — checks accept or reject readiness appropriately.
TC-09	Pilot navigation efficiency	PASS — documented 20% fuel-cost reduction.
TC-11	Engineer response to oxygen leak	PASS — repair response executed.
TC-12 / 13	Fuel depletion and successful arrival	PASS — failure and completion states triggered.
TC-14 / 15	Save and load mission	PASS — mission state persisted and restored.
TC-16	Load corrupt save file	PASS — invalid spacecraft data rejected.

8.1 Observations

- Encapsulation keeps resource changes within spacecraft operations.
- Crew roles influence the response to selected emergencies.
- Pre-launch validation prevents missions with insufficient resources or invalid crew.
- File validation protects the application from malformed persisted state.
- The test report records successful outcomes for the listed scenarios.

8.2 Applications

The project can be used as an educational example for object-oriented design, class relationships, polymorphic behavior, resource-state simulation, input validation and basic persistence. Its domain theme also provides an intuitive setting for demonstrating software modularity.

9. CONCLUSION, FUTURE SCOPE & REFERENCES

9.1 Conclusion

The Space Mission Simulator demonstrates the application of core Object-Oriented Programming concepts through a modular C++17 terminal application. It models spacecraft resources, destinations, crew specialization, mission stages and emergency events. The design uses encapsulation, abstraction, inheritance and runtime polymorphism, together with smart pointers, move semantics, exception handling and file I/O.

The documented test cases cover major mission workflows, boundary conditions and persistence operations. Overall, the project provides a practical academic demonstration of how OOPS concepts can be combined to build a structured and extensible simulation.

9.2 Future Scope

- Develop a graphical interface for mission setup and telemetry monitoring.
- Add richer destination and travel models.
- Introduce additional event types and configurable mission scenarios.
- Provide charts for fuel, oxygen and hull-integrity changes over time.
- Expand automated testing and reporting capabilities.

9.3 References

- Project source code and technical documentation: Space Mission Simulator OOPS Project Report (provided project document).
- ISO C++17 language and standard library concepts used in the implementation.
- GNU Make build utility documentation.

PROJECT SUBMISSION DETAILS

Student Details	Faculty Details
Name: _____\nUID: _____\nSection: _____\nSignature: _____	Submitted to: _____\nDesignation: _____\nDate: _____

— END OF REPORT —